

3.1 From Recurrence to Attention

A chain of states is a bottleneck. Attention gives every token a path of length one.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

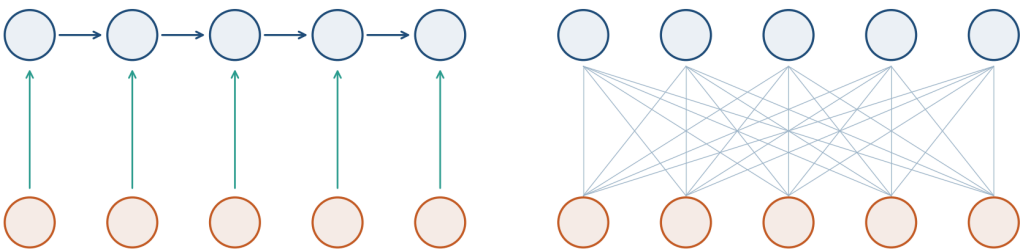
An RNN compresses the whole past into one vector. Token 1 can affect token 40 only by surviving 39 overwrites. **Attention** lets every token look at every other token in one step. That is the move from this week’s recurrences to the rest of the course.

1. The bottleneck

To answer “what does *it* refer to?” the model needs a path from *it* to a name that may be far left (or right). In an RNN that path is a chain. In attention it is one matrix of scores.

RNN: each state sees the past through a chain

Attention: every token can look at every token



Bahdanau et al. (2015) added attention on top of an encoder–decoder RNN so the decoder could reread the source. Vaswani et al. (2017) dropped the recurrence: **self-attention** is the layer.

Draw *The cat ... it* with 40 dots. RNN: 39 arrows in a line. Attention: one arrow from *it* to *cat*, plus arrows to everyone else, with **weights**. Note 3.2 turns those weights into softmax.

2. What you gain and what you pay

	RNN	Self-attention
Path length, token 1 to T	T	1
Parallel over time	no	yes
Cost in T	$O(T)$	$O(T^2)$ memory/time for full attention

Long documents (Week 7: efficiency) will force approximations. For this course’s default, full attention is the model.

Parallel over time means: given all embeddings X , you can form Q, K, V with matrix multiplies that a GPU loves. An LSTM must finish step $t - 1$ before step t . That, more than “transformers are magic,” is why pretraining scaled.

Write T and T^2 on the board for $T = 32, 512, 4096$. The path-length win is real; so is the memory bill. Sparse or sliding-window attention (Week 7) is what you do when T^2 no longer fits. You still teach full attention first.

Bahdanau attention is a decoder looking at encoder states: still recurrent, but the decoder is no longer forced to pack the source into one vector. Self-attention goes further: every token in the **same** sequence looks at every other token. That is the Week 3 default.

3. Teaching this note

30–40 minutes. Bottleneck picture, path-length table, then the $T = 4$ vs $T = 40$ cost arithmetic. Play **0:00–12:00** of 3Blue1Brown’s GPT visual intro (next-token prediction and the first glimpse of attention). Do **not** derive QKV here; that is note 3.2.

Minute plan: 8 min chain vs all-pairs; 10 min table and T^2 arithmetic; 8 min worked example; 10–12 min video. If you overrun, drop the Bahdanau history sentence, not the cost calculation.

4. Worked example

Sentence length $T = 4$ tokens: `cat`, `sat`, `on`, `it`.

RNN path from `cat` to `it`: 3 steps. Attention path: **1** score in the last row of a 4×4 matrix.

Full attention stores $T^2 = 16$ scores. For a 4k-token PDF, $T = 4096$, $T^2 \approx 1.68 \times 10^7$ scores **per head per layer**. If one score is 4 bytes, that is about 67 MB for **one** map, before batching and before stacking layers. That is the quadratic complaint.

GPU parallel: four tokens’ queries can be dotted with keys in one matmul. LSTM: four sequential cell calls.

Self-attention is still $O(T^2 d)$ for the scores if you materialize $QK^\top$. FlashAttention changes the **implementation**, not the fact that every pair can interact. For this course, write T^2 on the exam; mention fused kernels only as a footnote.

Decoder-only GPT still pays $O(T^2)$ in training. At decode time, caching keys and values makes each new token $O(T)$ against the past, not a full $T \times T$ from scratch. That is inference, not the training flop count.

Keep the slogan: **RNN path T , attention path 1, attention cost T^2** . If a student remembers only that, the week succeeded.

5. Where students get stuck

- Hearing “path length 1” and thinking attention is $O(1)$ compute (it is $O(T^2)$).
 - Mixing Bahdanau (attention **on** an RNN) with Vaswani (attention **instead of** an RNN).
 - Believing the RNN state “contains the whole sentence” in a lossless way.
-

6. Video

Watch [3Blue1Brown: But what is a GPT? Visual intro to transformers](#).

Pause when next-token prediction is a stack of vectors, and at the first all-to-all attention cartoon. Stop before the full QKV derivation if you are teaching 3.2 next; that math has its own video.

7. Practice

1. Why can a transformer GPU-batch a whole sentence’s updates at once, while an LSTM cannot?
2. A 4k-token policy PDF: what is the quadratic cost complaining about?
3. In one sentence, what information did h_t have to store that an attention row can fetch instead?
4. For $T = 8$, how many attention scores are in one head’s $T \times T$ map? For $T = 32$, how many? What is the ratio $32^2 / 8^2$?
5. An RNN of length $T = 50$ has path length 49 from token 1 to token 50. Write the attention path length. If each RNN Jacobian factor is 0.9, bound the RNN product 0.9^{49} vs attention’s single hop of 1.